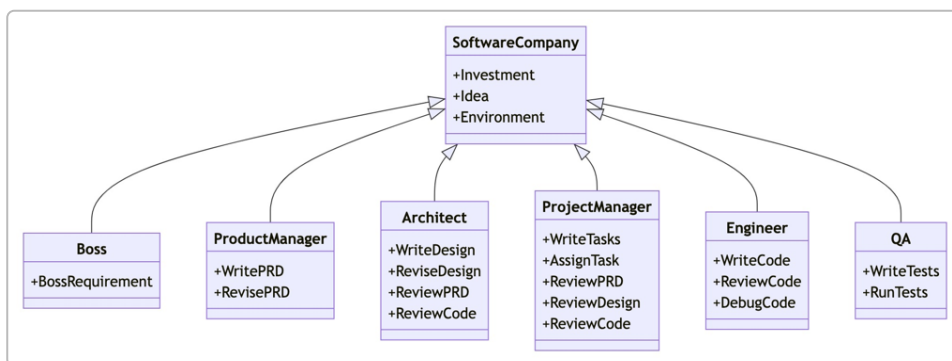


Next-Level Vibe Coding: Multi-Agent & Multi-Role Workflow

Introduction – From Solo Coder to AI Team Lead: When a *vibe coder* (an intrapreneur who uses AI to code without formal programming skills) reaches the point of **diminishing returns**, it's time to evolve. In early projects, a single AI chat might have been enough to build a basic app. But as complexity grows, you may find yourself fixing one issue only to break another, spending days in frustrating loops. The core problem is trying to do everything in one place – there's no separation of concerns or systematic validation. The solution? **Create a virtual AI development team** with multiple roles (AI agents) collaborating, much like a real software team. Recent advances in LLMs have made this feasible: today's AI agents can specialize in different tasks and work together following software engineering best practices ¹. This structured, multi-agent approach dramatically reduces errors and hallucinations by breaking complex tasks into clear subtasks handled by distinct "expert" agents ² ³. In short, you become a **team leader**, orchestrating several AI specialists instead of working alone.



Software Company as a Multi-Agent Team – An example from the MetaGPT framework. Distinct AI agents (e.g. Boss for business requirements, Product Manager, Architect, Project Manager, Engineer, QA) collaborate in sequence to turn a one-line idea into a complete project ¹.

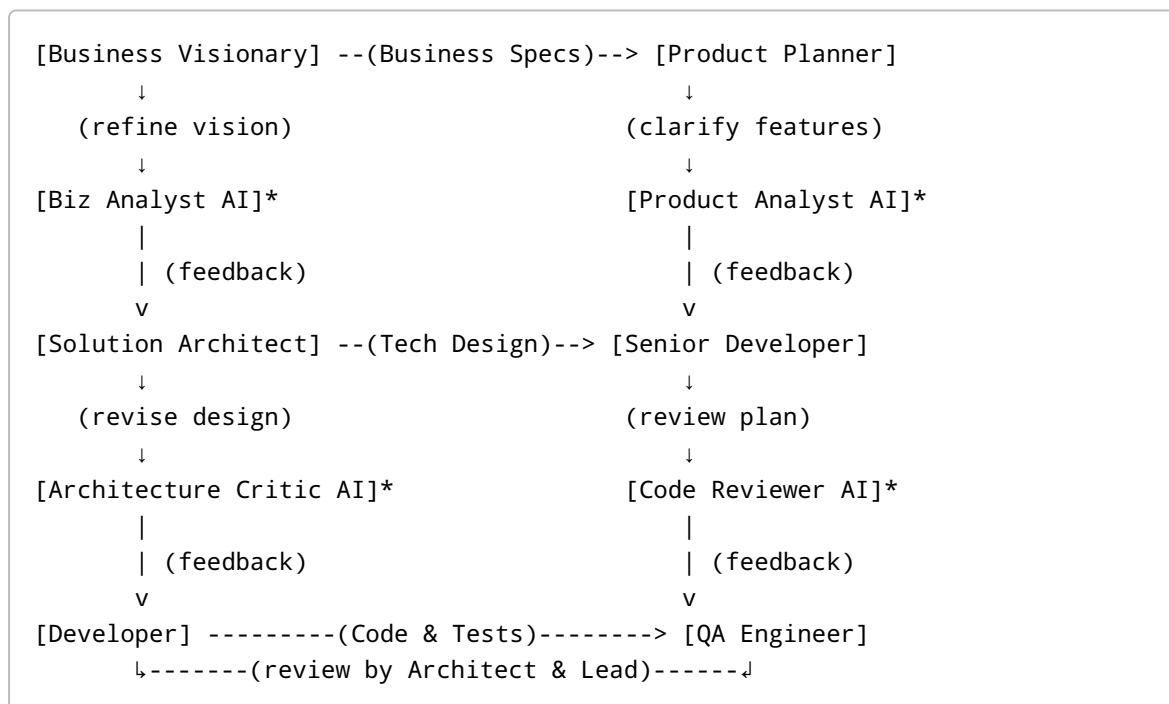
Overview of the Multi-Agent Workflow

In this next-level workflow, you will maintain multiple AI chat sessions – each isolated to a specific **role** in the software development life cycle. Think of it as managing a team of experts: one focuses on **business strategy**, another on **product design**, others on **architecture**, **implementation**, and **testing**. Each AI agent receives inputs from the previous stage and produces outputs for the next. All artifacts (documents, plans, code) are stored in version control, giving you traceability and the ability to revert or audit changes at each stage. The table below summarizes the primary roles and their outputs in the pipeline:

AI Role (Persona)	Key Responsibility	Main Output Artifact	Feeds Into
Business Visionary (Biz Lead)	Define business goals, value prop, success criteria, budget, scope.	Business Requirements & Vision Doc (non-technical)	Product Planner

AI Role (Persona)	Key Responsibility	Main Output Artifact	Feeds Into
Product Planner (Product Manager)	Define features and user stories from business goals; UX flows.	Product Requirement Document (PRD) – user stories, use cases, UI/UX flows	Solution Architect
Solution Architect (CTO/ Architect)	Design technical solution and architecture to implement PRD features.	Technical Design Specs – architecture diagrams, tech stack, component designs	Senior Developer
Senior Developer (Lead Engineer)	Plan implementation in detail; break down tasks, propose code structure.	Implementation Plan – task list, pseudocode, module specs	Developer (Coder)
Developer (Code Engineer)	Write the actual code according to the implementation plan.	Code Changes (feature branch, pull request diff)	QA Engineer & Code Review
QA Engineer (Tester)	Verify the functionality meets requirements; quality assurance.	Test Plans & Reports (test cases, results)	(Cycle back for fixes or finish)

This pipeline mirrors a real **agile team workflow** – from a high-level vision down to tested code. Below is an ASCII diagram of the flow, including hand-offs between roles and the iterative feedback loops:



(In the diagram, roles with an asterisk * are optional “reviewer” agents for quality control, explained later.)

In practice, the development flows **top-down**: The Business Visionary sets the direction, which is elaborated by the Product Planner, and so on. At each step, the specialist agent focuses only on its domain, producing a well-defined output (requirements doc, user stories, design spec, etc.). The next

agent then uses that output as the input context for its work. This assembly-line paradigm ensures a logical progression and clear hand-offs ³. If something is unclear or missing at any stage, you pause the flow and possibly loop back: e.g. if the Developer finds a missing detail in the design, you return to the Architect to fill that gap **before** coding. By dividing the problem into roles, you get cleaner outputs and avoid confusing one context with another. It also means you can apply **verification at each stage** – using either your own judgment or additional AI agents – to catch mistakes early.

Next, we'll break down each role in this virtual team, detailing their responsibilities, outputs, and how you can prompt an AI to fulfill that role. We'll also introduce persona names for each role to make this more tangible. Think of these AI agents as characters on your team, each with a specific job and personality tailored to that job!

Virtual Team Roles and Responsibilities

Now let's meet the **AI team**. Each role corresponds to an AI agent (an LLM session) with a clearly defined scope. We'll describe each persona's purpose and deliverables, along with example instructions (the *LLM role brief*) you would give to that agent. This brief sets the stage by explaining the project and the agent's specific mission. By structuring the prompts this way, you ensure each AI "team member" knows exactly what is expected from them and what to produce.

Role 1: The Business Visionary – *Defining the Why* (Business Lead)

This is the *CEO/Product Owner* of your virtual team – the agent that captures **what the business is trying to achieve**. The Business Visionary's focus is entirely non-technical: it clarifies the business model, target customers, value proposition, and success criteria of the project. In this role, you'll work with the AI to outline the core objectives and constraints of the product. You define things like the market problem being solved, how the product will make money (if relevant), what "success" looks like (KPIs or user adoption metrics), and any business constraints (deadlines, budgets, regulatory considerations).

Responsibilities: The Business Visionary produces a **Business Requirements & Vision Document**. This document should clearly state the product vision in plain language: for example, *"We are creating a mobile app for event planning that allows vendors and customers to connect. The business goal is to generate revenue through subscription fees... Success is defined by X number of active users in 6 months,"* etc. It may include simple diagrams of the user journey from a business perspective or flowcharts of how value and money flow through the system. At this stage, **clarity and simplicity** are key – it's meant to be understood by stakeholders and by the Product Planner (next role) without any technical jargon. All assumptions about what the end-user wants should be captured here. The output can also include a high-level **scope**: which features or use-cases are in scope for the first version (MVP) and which are out-of-scope (saved for later). Because everything the team does will trace back to these business requirements, accuracy and completeness here are vital. Version-control this document (e.g., commit it in a `/business` docs folder in a repository) so you have a history of changes in the vision.

LLM Brief for Business Visionary (Example): *Below is an example prompt you might use to instruct the AI in this role. Adjust details to your scenario.*

```
[System: You are "Visionary Vicky", an expert business strategist AI. Your job is to help define the business vision and requirements for a new software product.]
```

Project Overview: "We are building an AI-driven event planning app that connects vendors with clients."

Task: As the Business Visionary, clearly outline the business objectives, value proposition, target users, and success metrics for this project.

- Identify the problem being solved and why it's valuable.
- Describe the product at a high level and how it will meet customer needs.
- Define what success looks like (KPIs or business outcomes).
- List key constraints like timeline or budget, and any assumptions.

Output: A **Business Requirements Document** in Markdown that includes:

1. **Vision Statement** - What the product is and the gap it fills.
2. **Target Users/Market** - Who it's for.
3. **Value Proposition** - How it benefits them and any unique selling points.
4. **Business Model** - How it will generate revenue or sustain itself.
5. **Success Criteria** - Measurable goals (e.g., user count, revenue in 1st year).
6. **Scope & Constraints** - What's in/out of scope for the first version, any deadlines or budget limits.

Write in clear, non-technical language. Use bullet points or tables for clarity where appropriate. Include simple flow diagrams or ASCII art if it helps illustrate user or money flows.

(The above prompt establishes the context and instructs the AI to produce a structured business requirements doc. You would paste a prompt like this into your AI chat when starting the Business Visionary session.)

Role 2: The Product Planner – *Designing the What* (Product Manager)

Once the high-level vision is set, the baton passes to the Product Planner (think of this as your *Product Manager* or UX designer role). This agent's job is to translate business objectives into **tangible features and user stories**. The Product Planner takes the "why" from the Visionary and figures out **what the product must do** for the user. This is still largely conceptual and user-focused, not technical. It's about defining the *user experience, features, and requirements* in detail.

Responsibilities: The Product Planner produces a **Product Requirements Document (PRD)** or similar collection of user stories and feature specifications. This includes identifying the different types of users (user roles/personas) and detailing how each will interact with the system. For example, "As a customer, I can browse a catalog of vendors and send a booking request..." or "As a vendor, I can create a profile listing my services...". Each feature should be described along with acceptance criteria (conditions that will confirm the feature is working as intended). The Product Planner may also outline the navigation flow (screen-by-screen or page-by-page flow), create low-fidelity **mockups or flow diagrams** (ASCII art or Markdown mermaid diagrams can be used for simple wireframes or flowcharts), and map out how the user moves through the application. Crucially, the Product Planner does *not* decide how to implement these features technically – only what the features are and how they behave from the user's perspective. The output is written in clear language so that even a non-engineer (or our entrepreneur) can understand the intended behavior. This PRD will be handed off to the technical team (Architect next) and also serves as a reference for QA later to ensure the finished product meets the described behavior.

LLM Brief for Product Planner (Example): Once you have the business vision doc, you start a new AI session for the Product Planner. You might prompt it like this:

[System: You are "Product Planner Pete", a seasoned product manager and UX designer AI. You translate business goals into detailed product features and user stories.]

Context: We have a business vision for an "AI-driven event planning app connecting vendors with clients." (The business requirements from Visionary: [include a brief summary or the full Vision doc here].)

Task: As the Product Planner, produce a comprehensive Product Requirements Document (PRD) for this app.

- Identify all user types (e.g., Customer, Vendor, Admin) and write user stories for each (using "As a [user], I want [feature], so that [benefit]").
- For each user story, detail acceptance criteria (what must be true for the feature to be considered done).
- Sketch the primary user flows: how does a user sign up, search, book a vendor, etc. Use simple flow diagrams or step-by-step outlines.
- List the key screens/pages and what information or actions each should have (e.g., Home, Search Results, Vendor Profile, Booking Checkout, etc.).
- Prioritize features if necessary (MVP vs nice-to-have).

Output: A **markdown document** with clearly labeled sections:

1. **Overview** - Summarize the product and user goals.
2. **User Roles & Personas** - Who the users are.
3. **User Stories** - A list of user stories by role.
4. **Feature List** - High-level list of features to implement.
5. **Detailed Requirements** - For each feature, describe it in detail and provide acceptance criteria.
6. **User Flow Diagrams** - ASCII or text diagrams illustrating key workflows (e.g., user registration, booking process).
7. **Non-Functional Requirements** (if any) - e.g., performance needs, legal/privacy considerations.

Ensure the requirements align with the business vision. Keep language user-focused and avoid technical solution details (no mention of specific technologies or design patterns - that's for later).

This prompt guides the Product Planner AI to produce a thorough PRD. After it generates the output, you (as the project lead) should review it for completeness. Does it cover all the business goals stated by the Visionary? Is everything clear? If not, you can ask follow-up questions or edits in the Product Planner chat. Once satisfied, save this PRD (e.g., in a `/product` repository or folder). This document will be the *contract* of what needs to be built, which you then pass to the Solution Architect.

Role 3: The Solution Architect – *Designing the How* (CTO/Technical Architect)

Now we move into the technical realm. The Solution Architect is your **CTO agent**, responsible for figuring out *how* to implement the product requirements in an optimal way. This AI will translate the

Product Planner's user stories and features into a **technical blueprint**. Think of architecture as choosing the building design and materials before construction starts.

Responsibilities: The Solution Architect produces a **Technical Design Specification**. This often includes high-level architecture diagrams, a breakdown of the system into components or microservices, data models, API designs, and technology stack decisions. For example, the architect might decide: *"We will build a web application with a React frontend and a Python (Django) backend. We will use a PostgreSQL database to store data. The system will integrate with a payment API for transactions,"* etc. Each major feature from the PRD is mapped to a component or module in the design. The architect should address how different parts of the system interact (for instance, how does the mobile app talk to the server? What does the data flow look like when a booking is made?). Diagrams are extremely useful here – the AI can describe them or even use ASCII art or tools like Mermaid UML to sketch module relationships and data flows. Additionally, the architect defines **non-functional requirements** and how to meet them: e.g., security considerations, scalability (will the system handle 100k users?), performance targets, etc., as applicable to the project's scope. Importantly, the Technical Design Spec should be *feasible* and not overly theoretical – since our next step will be to implement it. Because our entrepreneur might not be a tech expert, this is where using a second AI as a reviewer becomes invaluable (we'll discuss the review process in the next section). For now, the architect's output needs to be clear enough for a senior developer to take each component and plan actual code.

LLM Brief for Solution Architect (Example): Start a new session for the Architect role. Provide the context of the PRD from the Product Planner. For example:

```
[System: You are "Architect Alice", a veteran software architect AI (CTO level). You design robust technical solutions from product requirements.]
```

```
Context: We have a Product Requirements Document for an "AI-driven event planning app." (Include the key points or attach the PRD here for reference.)
```

```
Task: As the Solution Architect, create a Technical Design Specification that will guide the engineering of this product. Your responsibilities:
```

- **Architecture Overview:** Describe the overall architecture (e.g., client-server, microservices, etc.), including the major components of the system (frontend, backend, database, third-party services).
- **Component Breakdown:** For each major feature or module (from the PRD), outline which component or service will handle it. Define the responsibilities of each component (e.g., "Authentication Service - handles user login/registration with JWT tokens").
- **Data Model:** Propose an initial database schema or data structures for storing key entities (users, vendor profiles, bookings, etc.).
- **APIs/Integration:** Specify any external APIs or integrations (e.g., payment gateway, mapping service) and how they'll fit in. Design internal APIs or endpoints as needed for the frontend-backend interaction.
- **Tech Stack:** Recommend technologies/frameworks for each part (frontend, backend, DB) considering the project needs (note any rationale, e.g., "Using Firebase for quick development of real-time chat").
- **Diagrams:** Provide simple architecture diagrams or descriptions (text-based is fine) to illustrate how components interact. For example, a request flow: "Client -> API Gateway -> Backend Service -> Database".
- **Non-Functional Considerations:** Briefly address security (e.g., auth

method, data protection), scalability (can it handle growth? any cloud services to use?), and deployment approach (will this be a web app, mobile app, etc., and how to deploy).

- **MVP Scope Tech Plan:** If the PRD has a lot of features, focus on an MVP (minimum viable product) implementation first - which parts will be built first and which can be stubbed or simplified initially.

Output: A **Technical Design Doc** in Markdown that covers the above points in sections. Include any necessary diagrams (ASCII art or pseudo-UML). Use clear, concise language (some technical terminology is expected here, but remember the next person reading might be an AI developer who will need clarity).

After this, the Architect AI will generate a technical plan. Given the complexity, it's normal if the first draft has some issues (LLMs might hallucinate technical solutions or make assumptions). This is where you should leverage a **second opinion** - we'll cover the *Architecture Critic* in the next section, who can review and validate this design. For now, assume Architect Alice's output is a structured blueprint for the system. Save it in version control (e.g., in a `/architecture` folder). The senior dev will soon need it.

Role 4: The Senior Developer - *Planning the Implementation* (Lead Engineer)

With a clear architectural blueprint, the Senior Developer (Lead Engineer) can now plan the actual work of coding. This agent acts like a **tech lead** or senior engineer who takes the high-level design and breaks it into an actionable implementation plan. If the Architect answered *"what is the system and how does it generally work?"*, the Senior Developer answers *"how do we build this system step by step?"*.

Responsibilities: The Senior Developer produces a **Detailed Implementation Plan** (or Technical Task List). This includes taking each component or module from the architecture and describing how to implement it, often in smaller tasks or stories that a coder can directly tackle. For example, if the architecture has a "User Auth Service", the Senior Dev might outline: *Task 1: Set up a new `auth-service` module using Django, Task 2: Implement JWT authentication with login/logout endpoints, Task 3: Integrate with front-end via API calls.* Essentially, this agent writes what could be considered an internal developer design: it might include pseudocode, class structures, or even small code snippets as examples for tricky parts. It should list dependencies (e.g., "we will use library X for payment integration - need to install it") and clarify any **implementation decisions** needed (like *"use OAuth2 for login via Google - thus implement using library Y"*). At this stage, it's all about **granularity** - making sure no feature is hand-waved, and everything the Developer will code is clearly specified. The Senior Dev might also establish **coding conventions** or project structure (e.g., "We will organize the code into these directories, follow PEP8 style for Python," etc.) to keep the project maintainable. The output is typically a **task breakdown** (possibly in bullet or numbered list form) and could be supplemented by sequence diagrams or further detail where necessary.

LLM Brief for Senior Developer (Example): Start a new chat for the Senior Developer role. Provide the context of the architecture design from the previous step. For example:

[System: You are "Senior Dev Dave", an expert software engineer and team lead AI. You plan implementation details from an architecture design.]

Context: We have a Technical Design for the event planning app (summarize or attach the Architect's output here). It specifies the architecture and

components.

Task: As the Senior Developer, create a **detailed implementation plan** for the engineering team to follow. Break the solution into clear development tasks. Specifically:

- **Task Breakdown:** List all the tasks or user stories required to build the system, organized by component or feature. (e.g., "Frontend: Implement homepage UI with search form; Backend: Create `/api/search` endpoint to query vendors...", etc.)
- For each task, provide details on **how** to implement it:
 - Mention specific functions, classes, or modules that need to be created.
 - Outline algorithms or logic in pseudocode if non-trivial.
 - Identify any external libraries or packages to use for that task (and ensure they align with the architecture's tech stack).
- Ensure the plan covers all features from the PRD and follows the architecture guidelines. If the architecture left some decisions open (e.g., how exactly to structure the database models), make those decisions now and document them.
- **Code Examples:** For complex parts, include short code snippets or examples (in the chosen language) to illustrate how to implement (for instance, an example model class definition, or a snippet of how to call an API).
- **Verification steps:** Suggest what kind of testing or validation each task should have (this helps QA later; e.g., "After implementing login, test that a user with correct credentials can obtain a token and access protected endpoints").

Output: A **Developer Implementation Plan** in Markdown:

- It should be structured (you can use a numbered list or headings for each component).
- Each item = a task or set of tasks with enough detail that a developer (or AI Developer agent) can start coding right away.
- Keep language concise and focused on **how to implement**. Reference the architecture doc for any needed clarifications if something is beyond scope.

Remember, you're writing this for a developer to use. It should be precise and unambiguous.

With this prompt, Senior Dev Dave will generate a step-by-step plan. It might be a long list of tasks, which is good – it shows every piece of work identified. As our *intrapreneur without coding expertise*, you might not fully understand all the technical specifics in this plan (and that's okay). This is another point where a **reviewer agent** can help: you could have the Architect or a second "Senior Dev Reviewer" agent go over this plan to ensure it aligns with the architecture and nothing critical is missing or incorrect. Often the act of detailing tasks will reveal prior omissions (e.g., the Architect might have forgotten a certain feature, and Senior Dev plan will surface the question). If discrepancies arise, you can loop back: update the Architecture doc or clarify with the Product Planner if needed, then adjust the plan. This iteration mimics a real dev team refining requirements and design before coding. Once the implementation plan looks solid, commit it (e.g., `/development_plan` docs).

Role 5: The Developer – *Building the Solution* (Coder)

At last, we arrive at writing actual code. The Developer role is the AI agent that will **write and modify the source code**. This is the only role that interacts directly with your codebase/repository. In human terms, this is the software engineer writing the code for each task, following the plan and design given by the earlier roles.

Responsibilities: The Developer's job is to **implement the tasks** from the Senior Developer's plan, one by one, in the codebase. This includes creating new files, writing functions, classes, tests, etc., as specified. Ideally, you will use a development environment that the AI can work in (for example, an AI coding tool or IDE integration like Cursor, or simply copy-pasting code back and forth from your editor). For each task or set of tasks, you instruct the Developer AI on *what to do*, based on the plan, and it will output code diffs or code blocks. **Crucially, before coding, ensure the Developer AI has all necessary information:** provide it the relevant parts of the plan, the architecture, and even the specific file context if needed. A great practice (as noted in the voice memo) is to ask *"Do you have everything you need to implement this?"* – let the AI double-check context. If it asks questions like *"Should I use technology X or Y here?"* then likely something in the plan or architecture was ambiguous. In such cases, pause coding and resolve the question (maybe by consulting Architect Alice again or making an executive decision). Once clarified, proceed with implementation.

The Developer should produce code output. You may have it output a unified diff or the full file content. After coding a piece, it's wise to run tests or at least have the Developer agent run through example usage (if your environment allows execution or you do it manually). When a segment of code is ready, treat it like a **pull request**: gather the diff of changes. **Before merging** these changes into the main codebase, you will invoke the earlier roles for a review (like a code review process). For example, have Senior Dev Dave or Architect Alice inspect the diff to see if it matches the plan and is high quality. The AI reviewers can often catch mistakes or suggest improvements in the code. Only after getting an approval from the reviewers do you then merge (commit) the code to the main branch. This staged commit process ensures the code is vetted, much like a real team would do via code reviews and approvals.

LLM Brief for Developer (Example): Now we create a session for the Developer. This role will be very interactive and iterative, often handling one task at a time. Here's how you might initiate it:

```
[System: You are "Coder Carl", an AI software developer. You write code based on provided designs and tasks, and you have access to the project repository files.]
```

```
Context: We have a detailed implementation plan for the event app (summarize relevant task or provide directly the plan for the feature you're about to implement). The current codebase status: [describe what files or scaffolding exists so far, if any].
```

```
Task: Implement the following feature as per the plan: **User Registration and Login** (for example). According to the plan:
```

- We need a signup page on the frontend and corresponding backend API endpoints ``/api/register`` and ``/api/login``.
- The backend should validate input, hash passwords, store user accounts in the database, and return a success or token.
- Use JWT for auth tokens as decided in architecture.
- There should be basic form validation and error messages on the frontend if

input is invalid.

Your job: Write the code required for this feature on both frontend and backend. Adhere to the architecture and plan details:

- Frontend: likely in React (JavaScript) - create a `Register.jsx` component with form, and a `login.jsx` or similar. Hook them into existing app routes if present.
- Backend: in Django (Python) - create an `auth` app/module. Implement `register` view to create a user (use Django's User model), and `login` view to authenticate (check password, generate JWT).
- Update any relevant settings or configs for JWT (e.g., secret key, token expiry).
- Ensure to follow coding conventions from the plan (for example, if the plan says to use Django REST framework, use it).
- Write unit tests for the new backend functions (if specified in plan or for good measure).

Check that you have all necessary information. If something is missing or unclear (e.g., "Do we already have a User model or should I create one?"), ask for clarification **before** writing code.

Output: Provide the code changes. Ideally, output them in a unified diff format or clearly indicate which file and where changes occur, so it can be reviewed easily. Ensure the code is well-formatted and includes comments where needed.

As you proceed with the Developer agent, you might implement feature by feature. Each time, review the code (yourself or via another agent). You'll loop: **Plan -> Code -> Review -> Adjust -> Commit** for each major feature. This is labor-intensive but keeps quality high. Remember to run the QA tests (next role) after implementing features, to catch any functional issues.

Role 6: The QA Engineer – *Ensuring Quality* (Tester)

The final critical role in the team is the QA Engineer – the agent responsible for **quality assurance and testing**. Even if the code was reviewed, we need to verify that the software actually works as intended and meets all the acceptance criteria from the Product Planner's user stories. This agent acts like a dedicated tester or QA team member.

Responsibilities: The QA Engineer creates and executes **test plans** to validate every feature. This includes writing test cases for each user story's acceptance criteria (e.g., "Test that a user can successfully register with valid details and cannot register with an email that's already taken," etc.), performing these tests (which could be manual steps described, or automated test scripts if within scope), and reporting any bugs or discrepancies. The QA agent might produce actual test code (like unit tests or integration tests in code form) to be added to the codebase, or it may outline testing procedures for a human or another tool to follow. Ideally, incorporate automated tests into your development: you can have the Developer agent write unit tests during implementation, but the QA agent ensures **completeness** of testing, covering edge cases that developers might miss. Another output from QA could be a **Test Report** – listing what was tested and the results (pass/fail), including any bugs found. If bugs are found, QA communicates those back to the Developer (this could mean opening a bug ticket or simply informing you to task the Developer AI with fixes). This creates a feedback loop: Developer fixes code -> QA retests until all tests pass.

LLM Brief for QA Engineer (Example): After some features are implemented (or for the whole app when near completion), start a QA session. For example:

[System: You are "QA Queen", a meticulous Quality Assurance engineer AI. You test the software to ensure it meets requirements and has no bugs.]

Context: We have an event planning app developed (features X, Y, Z are implemented as per the PRD). (Provide the list of features and any available test environment details, or even the codebase if possible.)

Task: As the QA Engineer, prepare a **Test Plan** and execute tests for the product:

- **Test Plan:** List test cases for each user story and feature from the PRD. Consider valid inputs, invalid inputs, edge cases, and security tests (e.g., ensure an unauthorized user cannot access protected data).
- For each test case, describe the steps to perform and the expected result.
- Mark each test as Pass/Fail based on the current implementation (you might need to infer the result from the code or ask me to run certain scenarios if needed).
- If any test fails or a bug is noticed, describe the bug in detail and ideally point to which part of the system might be causing it.
- Also perform some **exploratory testing**: think of scenarios not explicitly in requirements (e.g., performance with a large number of records, or UI responsiveness on mobile) and note any issues.
- If possible, write automated test scripts for critical parts (e.g., unit tests for the booking logic, or a simple script that simulates user actions).

Output: A **Test Report** in Markdown:

- Sections by feature or user story, listing test cases and their status.
- A summary of overall quality (e.g., "5 of 7 test suites passed; 2 issues found in the booking workflow").
- A list of **bugs or issues** discovered, with descriptions and severity.

Be thorough and unbiased - your goal is to identify any problem so it can be fixed before release.

The QA agent will generate test cases and likely identify problems if any. If issues are found, this is fed back into the development loop: you go back to Coder Carl (Developer) and fix those bugs, then perhaps re-run QA. Once QA reports all tests passed, congratulations - you have a high-confidence product ready to deploy!

Introducing Secondary Agents: Critics & Reviewers for Quality Control

In a traditional team, work products are often double-checked - code reviews, design reviews, etc. Your AI team can mimic this by having **paired agents** for crucial roles. The idea is to assign a *"critic"* or *"reviewer"* agent to evaluate the output of another agent. This is especially useful when you (the human) lack the expertise to judge the output's quality. For example, if you aren't comfortable assessing

whether the Architect's technical design is sound, you can deploy an **Architecture Critic** agent to do that for you.

How it works: after one agent produces an output, you give that output to a second agent whose role is to find flaws, ask questions, and provide suggestions. Then you feed those comments back to the original agent and ask it to revise its work. This back-and-forth continues until the two agents reach a consensus or at least the output is significantly improved. Not only does this catch mistakes, it also reveals if your requirements are unrealistic or unclear. *If two knowledgeable AI agents can't agree on a solution, it's a signal that the problem definition might need adjustment.* This iterative "AI peer review" greatly reduces the chance of hallucinations or logical errors cascading down the pipeline ³. In essence, you create a mini debate or pull-request style review between AIs.

You can set up a reviewer for any role, but the most value comes from technical stages where precision matters. Here are the recommended secondary roles (the "Review Team"), along with fun persona names and their purposes:

- **Business Analyst (Biz Analyst Bob)** – Reviews the Business Visionary's output. Checks if the business plan is coherent, if goals align with the problem, and if anything is missing (e.g., "Did you consider how to retain users?"). Since the business doc is more about clarity than correctness, you may or may not need this review. It's optional for extra polish.
- **Product Analyst (Product Critic Chloe)** – Reviews the Product Planner's PRD. Verifies that all business requirements are addressed by features, looks for any missing edge cases in user stories, and ensures the PRD is understandable. Essentially a second pair of eyes on the feature list to ensure completeness.
- **Architecture Critic (Architect Reviewer Rex)** – Reviews the Solution Architect's design. This agent should have a strong technical background (you can even prompt it with a slightly different specialty or use a different LLM model for diversity). It will scrutinize the proposed architecture for feasibility: "Are the chosen technologies suitable? Will the design scale? Are there security vulnerabilities? Is anything over-engineered or under-specified?" ⁴. It might catch, for example, that the design forgot a logging/monitoring component, or that a certain integration is unrealistic. The critic's feedback is fed to the Architect agent for revision. This loop can continue until the architecture is solid.
- **Senior Dev Reviewer (Lead Engineer Erin)** – A second Senior Developer agent or even the Architect agent acting again, to review the Implementation Plan. This reviewer ensures the plan covers all parts of the design, follows best practices, and doesn't include any logical gaps. They might point out, say, "We haven't accounted for email verification in the auth tasks" or "Maybe use library Z instead of writing Y from scratch, as it will save time." Again, feed those comments back to Senior Dev Dave to update the plan.
- **Code Reviewer (Code Critic Calvin)** – You can have an agent dedicated to reviewing code outputs from the Developer. In practice, we suggested using the Senior Dev or Architect as the code reviewer (to mimic how tech leads review code). Whether it's a distinct persona or re-using one of the above, ensure that every significant code change gets a review pass. The reviewer should check for correctness, adherence to the design, and readability/maintainability of code. They might catch bugs or suggest refactoring. Only after the code review agent is satisfied do you proceed to merge.
- **Test Auditor (QA Reviewer Quincy)** – (Optional) If you want, you can even have a reviewer for the QA stage – for example, an agent that double-checks the test cases for coverage. However, if the QA agent is doing a good job and you as the manager are reviewing the test results, this may be unnecessary. Usually, QA is the final check itself.

Using these secondary agents does incur more AI usage and coordination effort, but it *significantly boosts the reliability* of the final product. You are effectively simulating a full **peer review workflow**. Research has shown that such multi-agent role specialization with critique loops can eliminate many errors that a single-pass approach would miss ⁵ ⁶ .

Tip: To get the most out of reviewer agents, consider using *different LLM models* or at least slightly different prompting styles for them. For example, you might use GPT-4 for the main agents and use another instance of GPT-4 or a model like Claude or others for the critics, or even GPT-4 with a prompt that makes it act extra skeptical. The diversity in perspective can reveal different issues (just like two human engineers will think of different things). You then reconcile the outputs.

When setting up a critic agent, your prompt would be similar to the main role but explicitly ask it to analyze another agent's output. For instance, for Architecture Critic: *"You are an AI software architect tasked with reviewing a proposed system design. Here is the design document: [...] Please analyze it and list any potential issues, omissions, or improvements. Be as critical as possible."* Then take that list of critiques and present it to the original Architect agent with a prompt like *"Please address the following review comments in your revised design:"*. This iterative process may take a couple of rounds, but usually converges on a much-improved design.

Practical Implementation Tips for Your AI Team Workflow

Implementing this multi-agent workflow requires some discipline and setup, but here are some tips to make it smoother:

- **Use Separate Sessions or Windows:** Keep each role in a dedicated chat or AI session. This maintains context isolation. Tools like Cursor or multi-tab browser with ChatGPT can help. Name each session by role (e.g., "ProjectX – Business Visionary", "ProjectX – Developer") so you can easily switch.
- **Version Control Each Stage:** As mentioned, use Git or another version control system to track outputs. You might even create separate repositories or at least folders for different stages:
 - e.g., `project-business-docs/` for Vision and business docs,
 - `project-product-design/` for PRD and user stories,
 - `project-architecture/` for design specs,
 - and of course your main `project-code/` repo for code.

Commit each agent's deliverable with a clear message. This not only keeps you organized but also lets you rollback if an AI goes off-track. - **Communicate Context Clearly:** When handing off from one agent to the next, always provide the necessary context from previous outputs. You don't have to show the entire document if it's huge (summaries can work), but err on the side of giving more context to avoid misunderstandings. A lack of information is poison for AI productivity – it may start guessing and hallucinating. So, copy relevant sections from the Vision doc into the Product Planner's prompt, the PRD into the Architect's prompt, and so on. - **Stay In Charge of the Flow:** As the human in the loop, you act as the project manager or Scrum Master. Monitor each agent's output. If something looks off to you (even if you're not an expert, trust your intuition or logic), don't blindly accept it. Pause and question it, or engage a reviewer agent. It's easier to correct upstream in the design than downstream in code. - **Iterate in Small Steps:** Especially on the coding side, implement and test in small increments. Don't attempt to have the Developer agent code the entire application in one go. Start with a basic skeleton (maybe a "Hello World" deployment to ensure all pieces are connected). Then implement feature by feature. This aligns with agile principles (increments) and helps localize any errors, making debugging easier. It also ensures that the context given to the Developer agent is manageable and within token limits. - **Leverage Testing Throughout:** Encourage writing tests not just at QA stage but during

development. For instance, after the Senior Dev plan, you could have a brief where that agent (or the Developer agent) also generates unit test stubs alongside each task. This ensures the developer writes code with testability in mind. Automated tests can be run whenever possible to quickly catch regressions. - **Tools & Automation:** While doing this manually gives you a great understanding, be aware that there are emerging frameworks to automate multi-agent orchestration. For example, **MetaGPT** (as cited earlier) is a framework that essentially implements a lot of this workflow automatically ¹. It can take a one-line project idea and produce a whole repository by simulating a product manager, architect, engineers, etc. There are also tools built on **LangChain** and other libraries that let you script multi-agent interactions ⁷ ⁵. You might explore these once you grasp the manual process – they could save time on future projects. However, use caution: fully automated systems might not yet handle every nuance of *your* specific project, so oversight is still needed. Think of them as junior project managers that still need supervision. - **Scalability of the Approach:** Initially, running 5–10 AI agents might seem costly or slow. But consider that a year or two ago, doing this would require hiring an entire team (hundreds of thousands of dollars in salary) – now you might accomplish it with a few dollars of API calls (anecdotally, MetaGPT reports generating a full project for about \$2 in API costs ⁸). Still, optimize where you can: not every project needs every role fully fleshed out. For a simple app, you might combine Business and Product planning steps (since you can manage that overhead). Use the roles that add value for the complexity you have. As you get comfortable, you'll develop a sense of which stages to emphasize. - **Security and Other Non-Functional Roles:** As you extend this approach, you can incorporate specialized roles like a **Security Auditor AI** (to review the design and code for security flaws), a **DevOps/Deployment AI** (to create Docker scripts, CI/CD pipelines, etc.), or a **Project Manager AI** (to track tasks, maybe a Trello board of what's done/left). These map to “enabling teams” or specialists in real organizations. It might not be necessary on day one, but for a production-grade project, these considerations matter. For instance, a Security AI could scan code for vulnerabilities (or at least ensure OWASP Top 10 issues are addressed). A Deployment AI could automate cloud setup. Each would have their own brief and would feed back into the main dev loop with recommendations. - **Team Topologies Note:** This whole method essentially creates a *stream-aligned team* (one focused on delivering a product) that exists entirely in your head + AI. You as the intrapreneur are the hub that ties the team together. In *Team Topologies* terms, you might later interact with “platform teams” (maybe an AI that helps with cloud platform issues) or “enabling teams” (like the security role) ⁴, but initially your AI team is self-contained to deliver the product. Keep the *team-of-agents* nimble; you can always add more roles if needed, but each added agent is also added complexity.

Conclusion: Managing Your AI Software Team

Transitioning from a lone vibe coder to an **AI team lead** is a significant step up. It requires a shift in mindset – you are no longer “writing code with AI help,” you are **orchestrating a collaborative process**. This guide mapped out a comprehensive workflow with distinct AI roles mirroring a mature software development life cycle. By dividing responsibilities among specialized agents, you gain clearer outputs, better quality control, and a structure that can tackle bigger, more complex projects than a single chat ever could.

Expect a learning curve: at first, it might feel like overkill to write a vision doc or a PRD with an AI if you just want to hack together an app. But as you experienced, without structure, complexity becomes overwhelming and progress stalls. By investing time upfront in planning (with the Business, Product, and Architecture agents), you *save exponential time later* by avoiding missteps and rewrites. Each role's output serves as a safety net for the next – catching misunderstandings and mistakes early. The inclusion of reviewer agents further amplifies this, acting as your AI “colleagues” who double-check everything.

Many have found that once they get the hang of this multi-agent approach, their productivity and the reliability of the AI-generated code soars ⁶ ⁹. You as a non-coder can successfully steer the development of quite sophisticated software, essentially by **managing an AI-driven Scrum team**. You'll also start to develop an intuition for software engineering concepts by seeing how each role thinks and what it produces – in a way, it's a crash course in software development practices, with AI as both your teacher and workforce.

Finally, remember that this workflow is flexible. You can tailor the roles to your project's needs. Give your AI team fun names, treat them as real teammates, and enjoy the process – you might be surprised at how motivating it is to “work with a team” even if it's an imaginary AI team. As the technology evolves (and it's evolving rapidly), such AI teamwork will only become more seamless and powerful. By getting in early, you're effectively learning how to be a manager of AI specialists – a skill that will be incredibly valuable in the future.

Next Steps: Try this approach on a small sample project. For instance, task your team to create a simple To-Do list web app. Go through each role's motions, even if abbreviated: Visionary defines why this to-do app is unique, Product Planner writes a few user stories (add task, complete task, etc.), Architect designs a basic 3-tier app, Senior Dev outlines tasks, Developer writes the code, QA tests it. This could be done in a day. Through that exercise, you'll refine your prompting techniques and get a feel for the cadence of hand-offs. Then you'll be ready to tackle your real project with confidence, knowing you have a scalable workflow and a trusty AI team by your side. Good luck, and happy coding with your AI crew!

Sources: The concept of multi-agent AI teams is inspired by recent frameworks like *MetaGPT*, which assign roles like Product Manager, Architect, Developer, and QA to LLM agents to collaboratively build software ¹ ⁵. This structured approach, leveraging standard operating procedures from human teams, has been shown to reduce errors (avoiding “cascading hallucinations”) and improve the quality of AI-generated code ² ³. By mirroring proven software engineering workflows, we harness the strengths of LLMs in a controlled, purposeful manner – essentially running an AI-powered software company on your desktop ¹⁰.

¹ ⁸ **MetaGPT: The Multi-Agent Framework | MetaGPT**

https://docs.deepwisdom.ai/main/en/guide/get_started/introduction.html

² ³ ⁴ ⁵ ⁶ ⁹ **MetaGPT-style Software Team Agents: Foundations, Architecture, Applications, and Performance Trends**

<https://atoms.dev/insights/metagpt-style-software-team-agents-foundations-architecture-applications-and-performance-trends/7e48a158cab643e4b8ea7157286a92f2>

⁷ **[PDF] MetaGPT: Meta Programming for a Multi-Agent Collaborative ... - arXiv**

<https://arxiv.org/pdf/2308.00352>

¹⁰ **GitHub - FoundationAgents/MetaGPT: The Multi-Agent Framework: First AI Software Company, Towards Natural Language Programming**

<https://github.com/FoundationAgents/MetaGPT>